EEG-Based Emotion Recognition for Real World Application

Abstract

This study investigates EEG-based emotion recognition under constraints typical of consumer devices: few channels, high noise, and limited compute. We design a modular pipeline that standardizes preprocessing, feature extraction, training, and evaluation while preserving reproducibility through explicit option sets. Using the DEAP dataset, we run 1176 configurations spanning classical and deep models, multiple feature families, valence/arousal classification bins at 9-, 5-, and 3-class resolutions, and regression targets. Achieving up to 0.89 macro-F1 in 5-class classification, the strongest configurations highlight robust settings for practical deployment.

The strongest configurations for 9- and 5-class classification converge on clean preprocessing, differential entropy (DE) features, and a CF-CNN classifier with the full 32-channel montage. In contrast, the 3-class setting is best served by power spectral density (PSD) features paired with a tree-based model, reflecting the coarser label structure. Regression remains weak across configurations (low R2 even at best RMSE), indicating that discrete classification targets are currently more reliable. Low-channel analysis reveals two practical operating points: at <= 4 channels, a minimal temporal montage with ICA cleaning and DE features offers the most robust tradeoff; at <= 8/12 channels, an optimized lateral montage with clean preprocessing and DE features becomes the most stable option for 3-class classification.

Overall, the results provide a defensible set of configurations for low-channel, noisy deployment and demonstrate the value of a modular, reproducible pipeline for systematic comparison of EEG emotion recognition techniques. The application prototype anchors the study in a real-world deployment theme by validating these configurations with end-to-end integration from streaming input to mobile feedback, while quantitative evaluation remains centered on the pipeline.

Contents

1. Introduction

Electroencephalography (EEG) is a method that records the brain's spontaneous electrical activity. EEG is a non-invasive, portable technique with high temporal resolution, and it is used in various brain-computer interface (BCI) applications. Emotion recognition is the process of inferring human emotion from various sources of information, including physiological and physical signals [1], [2]. It has been shown to have wide application prospects in various fields, e.g., psychology, medicine, recreation, etc. [2]. In practice, EEG emotion recognition can enable mental health apps for stress detection or gaming systems with adaptive difficulty. Owing to the cerebral nature of emotions, EEG has been shown to be a strong signal source for emotion recognition.

Using machine learning (ML) and deep learning (DL) models, EEG-based methods achieve higher accuracy in emotion recognition than methods based on other signal sources, e.g., electrocardiography (ECG), galvanic skin response (GSR), eye tracking (ET), speech, and facial images [2], [3], [4].

Commercial EEG devices for emotion recognition have not become widespread yet. Attempts have been made to use currently available consumer-grade EEG devices, general-purpose or designed for tasks like meditation assistance, for emotion recognition by combining them with existing ML and DL models [5]. These models, though achieving strong results in the lab, have shown limitations when working with commercial devices. In a real-world setting, they struggle to address the following challenges:

1. Few channels.

   Commercial EEG devices have a limited number of channels, which constrains the performance of models trained on full-channel data [5].

2. Highly noisy environment.

   Environmental noise originating from electromagnetic signals that are ubiquitous in daily life might severely impair model performance, reducing the signal-to-noise ratio (SNR) of the EEG signals [1]. Also, wet electrodes, which collect higher-quality EEG signals, might not be available because they cause more discomfort than dry electrodes, further reducing the SNR.

3. Limited computational resources.

   A portable, consumer-grade device might have limited computational capacity and power supply. Based on these criteria, existing ML and DL models can be critically evaluated to guide the development of a practical EEG emotion-recognition application.

Although current research has provided partial solutions by proposing techniques that address these problems, a comprehensive study of an application that addresses all these issues for real-world deployment is still needed. To address these gaps, this study develops a modular pipeline for systematic evaluation and a prototype app for real-time deployment, with the specific goals of (i) find a functional technique stack that works well with few channels, noisy environments, and resource constraints and (ii) develop a prototype application that applies the techniques .

Building on these goals, the Methodology section first details the modular pipeline for reproducible technique comparison and then describes the application prototype that embeds the selected configurations in a real-world workflow. The Results section quantifies pipeline performance across the enumerated configurations, directly informing the deployment choices. Finally, the Discussion interprets these findings in the context of the deployment constraints, while Limitations and Future Directions outline paths for refinement.

2. Methodology

EEG-based emotion recognition can be performed by various combinations of techniques. We define a general pipeline for the procedural implementation of data loading, data preprocessing, feature extraction, and model training and evaluation. These techniques affect recognition performance in different ways. Ordering them into this pipeline facilitates the optimization, comparison, and selection of the available techniques for our specific goal.

Using the selected stack of techniques, we build a prototype application that simulates EEG signal collection, classification, and real-time streaming in real-world scenarios. The subsections below first detail the pipeline components and evaluation setup, then describe the application workflow that consumes the selected configurations, ensuring a clear link from systematic evaluation to practical deployment.

2.1. Machine Learning Pipeline

2.1.1. Design

The pipeline adopts a clean modular design with minimal coupling between modules, allowing different techniques and parameter sets to be adopted. As shown in Figure 1, it is organized as a staged workflow with five main modules: (i) Preprocessor , (ii) Feature Extractor , (iii) Model Trainer , (iv) Pipeline Runner , and (v) Result Explorer . The first three modules make up the main body of the pipeline, each providing validated outputs for downstream modules and producing persisted artifacts with explicit parameters and atomic writes. This modular structure directly supports the study's goal of identifying robust configurations under real-world constraints, as it enables exhaustive enumeration and comparison.

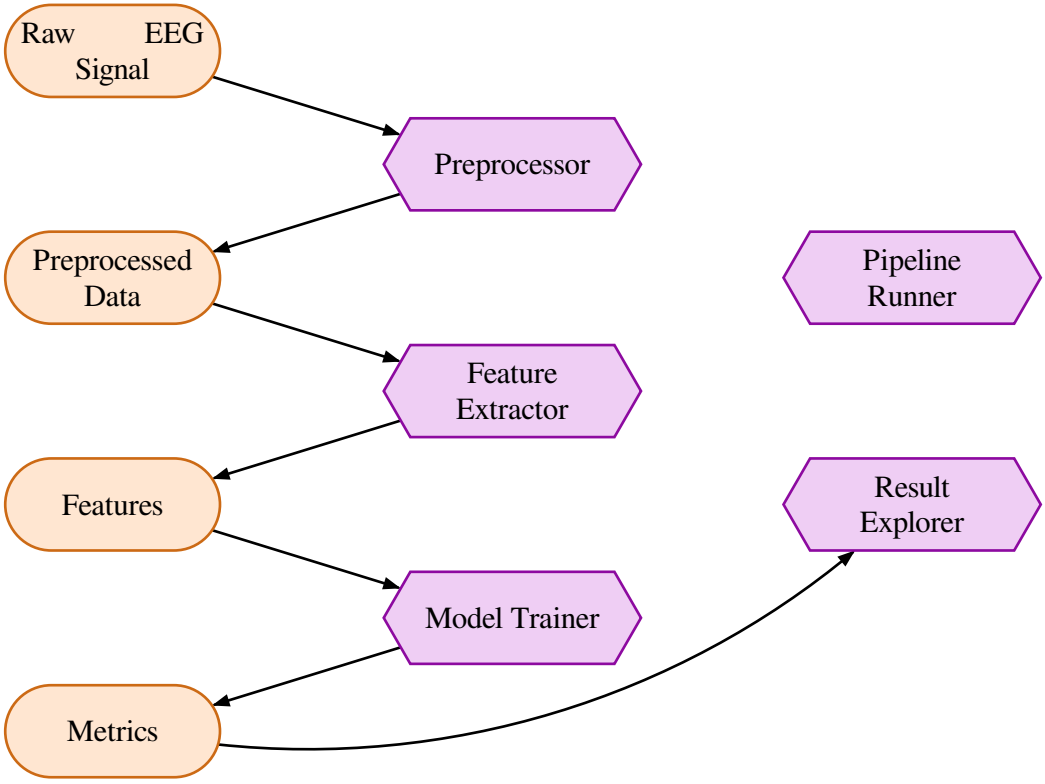

Figure 1: EEG Emotion Recognition Development Pipeline

2.1.2. Data Source

Many datasets can be used as the data source for the pipeline; among them, large, public datasets are considered the best-tested options because of their proven quality and reproducibility-friendly transparency [3], [5]. For the implementation of our pipeline, we choose the Database for Emotion Analysis using Physiological Signals (DEAP) [6]. In this study, we use only the EEG modality and two of the four labels (valence and arousal), excluding dominance, liking, peripheral physiology, and facial video.

The Database for Emotion Analysis using Physiological Signals (DEAP) represents a multimodal repository developed by Koelstra et al. to advance empirical research in affective computing and physiological-based emotion recognition. This dataset has established itself as a foundational instrument for systematically investigating emotional responses through objective psychophysiological measurements.

DEAP contains recordings from 32 participants who viewed 40 one-minute music video excerpts, meticulously curated to evoke diverse emotional reactions across the valence-arousal spectrum. Post-stimulus subjective assessments were acquired via continuous self-report scales measuring:

- Valence (unpleasant to pleasant)
- Arousal (calm to excited)
- Dominance (perceived control)
- Liking (subjective preference)

The dataset also integrates three concurrent data streams:

1. Electroencephalography: 32-channel EEG recorded at 512 Hz utilizing the Biosemi ActiveTwo system, subsequently downsampled to 128 Hz and preprocessed for artifact attenuation according to the International 10-20 montage.
2. Peripheral Physiology: Eight-channel peripheral signals encompassing galvanic skin response, blood volume pulse (photoplethysmography), respiration rate, skin temperature, and electromyography from zygomaticus and trapezius musculature.
3. Facial Video: Frontal facial recordings captured at 50 Hz (available for 22 participants) to facilitate visual expression analysis.

2.1.3. Tools

The pipeline is implemented in Python 3.12, using `uv` for package management and `Git` for version control, with the following core dependencies:

- `MNE-Python` for BDF ingestion, montage handling, filtering, and epoching.
- `NumPy` and `pandas` for numerical transformations and tabular feature assembly.
- `scikit-learn` for classical models and evaluation metrics, and `PyTorch` for neural models.
- `joblib` for serialized artifacts, with `matplotlib`, `seaborn`, and `Jupyter` supporting analysis and visualization.

2.1.4. Modules

The pipeline is composed of five modules that form a strict data flow from raw DEAP recordings to ranked experiment results. Each module exposes lightweight option objects that capture the parameters required for reproducible runs and deterministic artifact paths. This explicit parameterization ensures that configurations can be directly exported for deployment in the application prototype, maintaining consistency from evaluation to real-world use.

2.1.4.1. Preprocessor

Preprocessor loads raw DEAP recordings, isolates EEG channels, applies filtering/re-referencing (and optional ICA), epochs and resamples each trial, enforces canonical channel/trial order, and writes subject-level and trial-level artifacts with metadata for downstream stages.

The configuration for the preprocessor module is declared with a `PreprocessingOption` object, as defined in Table 1.

Parameter	Type	Definition
name	`str`	Stable identifier used in registries and artifact path s.
root_dir	`str | Path`	Root directory under `data/DEAP/generated/` for preprocessing outputs.
preprocessing_method	`PreprocessingMethod`	Callable `(raw: mne.io.BaseRaw, subject_id: int) -> np.ndarray` returning `(trials, chann els, samples)` with preprocessing applied.

Table 1: PreprocessingOption

2.1.4.2. Feature Extractor

Feature Extractor slices each trial into sliding windows, computes per-window features for the selected channels, and stores the resulting feature frames and baseline metadata under the preprocessing root.

The feature extraction configuration is composed from `ChannelPickOption`, `FeatureOption`, `SegmentationOption`, and `FeatureExtractionOption` objects (Table 2, Table 3, Table 4, Table 5). `FeatureExtractionOption` also derives a deterministic name and binds channel selection into the extractor callable.

Parameter	Type	Definition
name	`str`	Stable identifier used in option names and artifact paths.
channel_pick	`list[str]`	EEG channel names to retain; each must belong to the Geneva 32-chann el list.

Table 2: ChannelPickOption

Parameter	Type	Definition
name	`str`	Stable identifier used in option n ames and artifact paths.
feature_channel_extraction_me thod	`FeatureChannelExtractionMe thod`	Callable `(trial_data: np.nda rray, channel_pick: list[s tr]) -> np.ndarray` that map s one segment to a feature tenso r.

Table 3: FeatureOption

Parameter	Type	Definition
time_window	`float`	Window length in seconds; must be positive and no longer than the baseline i nterval.
time_step	`float`	Step size in seconds; must be positive and no larger than `time_window`.

Table 4: SegmentationOption

Parameter	Type	Definition
preprocessing_option	`PreprocessingOption`	Preprocessing configuration; see Table 1.
feature_option	`FeatureOption`	Feature definition; see Table 3.
channel_pick_option	`ChannelPickOption`	Channel selection; see Table 2.
segmentation_option	`SegmentationOption`	Sliding-window configuration; see Table 4.

Table 5: FeatureExtractionOption

2.1.4.3. Model Trainer

Model Trainer converts extracted feature frames into datasets, applies target encoding and scaling, constructs training loops or sklearn estimators, and writes metrics and model artifacts for each run.

Training is configured by a chain of options: `BuildDatasetOption`, `TrainingDataOption`, `TrainingMethodOption`, `TrainingOption`, `ModelOption`, and `ModelTrainingOption` (Table 6, Table 7, Table 8, Table 9, Table 10, Table 11).

Parameter	Type	Definition
target	`str`	Label column to predict (e.g., `valence`, `arousal`).
random_seed	`int`	Seed used for reproducible train/test splits.
use_size	`float`	Fraction of segments retained for the experiment (0 < use_size <= 1).
test_size	`float`	Fraction of retained segments used for the test set (0, 1).
target_kind	`TargetKind`	Literal `regression`, `classification`, `classification_5`, or `classification_3` defining target encoding.
feature_scaler	`Literal["none","standard","minmax"]`	Scaling strategy applied per segment before training.

Table 6: BuildDatasetOption

Parameter	Type	Definition
feature_extraction_option	`FeatureExtractionOption`	Feature configuration for loading segment frames; see Table 5.
build_dataset_option	`BuildDatasetOption`	Dataset construction knobs; see Table 6.

Table 7: TrainingDataOption

Parameter	Type	Definition
name	`str`	Stable identifier used in option names and artifact paths.
target_kind	`TargetKind`	Target type expected by the training loop.
backend	`Literal["torch","sklearn"]`	Training backend selection.
epochs	`int | None`	Number of epochs for torch backends; ignored by sklearn.
batch_size	`int | None`	Batch size for torch backends; ignored by sklearn.
optimizer_builder	`OptimizerBuilder`	Callable `(model: nn.Module) -> Optimizer` for torch backends.
criterion_builder	`CriterionBuilder`	Callable `() -> nn.Module` returning the loss function.
num_workers	`int`	DataLoader worker count.
pin_memory	`bool`	Whether to pin DataLoader memory for CUDA.
drop_last	`bool`	Whether to drop the last incomplete batch.
device	`Literal["cpu","cuda"]`	Device used for torch training.
batch_formatter	`BatchFormatter`	Callable `(features, targets, device, target_kind) -> (Tensor, Tensor)` that reshapes batches.
train_epoch_fn	`TrainEpochFn`	Callable for one training epoch over a DataLoader.
evaluate_epoch_fn	`EvaluateEpochFn`	Callable for one evaluation epoch over a DataLoader.
prediction_collector	`PredictionCollector`	Callable that gathers predictions/targets for reporting.
fit_kwargs	`dict[str, Any] | None`	Extra kwargs passed to sklearn estimators.

Table 8: TrainingMethodOption

Parameter	Type	Definition
training_data_option	`TrainingDataOption`	Loaded dataset configuration; see Table 7.
training_method_option	`TrainingMethodOption`	Training loop configuration; see Table 8.

Table 9: TrainingOption

Parameter	Type	Definition
name	`str`	Stable identifier used in option names and artifact paths.
model_builder	`ModelBuilder`	Callable `(*, output_size: int) -> nn.Module` for torch or `() -> BaseEstimator` for sklearn.
target_kind	`TargetKind`	Target type supported by this model.
backend	`Literal["torch","sklearn"]`	Model backend selection.
output_size	`int | None`	Output dimension for torch models; `None` for sklearn.

Table 10: ModelOption

Parameter	Type	Definition
model_option	`ModelOption`	Model configuration; see Table 10.
training_option	`TrainingOption`	Dataset + training configuration; see Table 9.

Table 11: ModelTrainingOption

2.1.4.4. Pipeline Runner

Pipeline Runner enumerates the Cartesian product of all registered options, invokes each module in order, and ensures artifacts are written for every valid combination. All parameters accept an `OptionList`, a sequence, or a single option; inputs are normalized to `OptionList` internally to keep orchestration deterministic.

Parameter	Type	Definition
preprocessing_options	`OptionList[PreprocessingOption]`	Preprocessing configurations to run.
channel_pick_options	`OptionList[ChannelPickOption]`	Channel selection configurations for feature extraction.
feature_options	`OptionList[FeatureOption]`	Feature method configurations.
segmentation_options	`OptionList[SegmentationOption]`	Sliding-window configurations.
model_options	`OptionList[ModelOption]`	Model architecture/estimator configurations.
build_dataset_options	`OptionList[BuildDatasetOption]`	Dataset construction configurations.
training_method_options	`OptionList[TrainingMethodOption]`	Training loop configurations.

Table 12: run_pipeline

2.1.4.5. Result Explorer

Result Explorer loads stored run artifacts from `results/`, builds a searchable results table, filters by target/target_kind, and ranks runs by chosen metrics. It is used by the notebook reports and the streaming mock app to evaluate trained pipelines.

Parameter	Type	Definition
results_root	`Path`	Root directory containing `results/<target>/<target_kind>/<timestamp>/` runs; `load_default_results_table` resolves this from `RESULTS_ROOT`.

Table 13: load_results_table

2.1.5. Implementation

Concrete option registries and parameterizations are instantiated in the codebase to enable reproducible experiment enumeration. The following entries define the actual configurations used by the pipeline.

The options are mainly extracted from a curated list of published studies, as listed in Table 14, and supplemented by trivial or common baseline choices. This curation ensures that the pipeline tests techniques with proven efficacy while extending them to address the deployment constraints introduced earlier.

Year	Title	Author	Citation Count
2003	A Neural Mass Model for MEG/EEG: Coupling and Neuronal Dynamics [7]	David and Friston	529
2005	Interictal to ictal transition in human temporal lobe epilepsy: insights from a computational model of intracerebral EEG [8]	Wendling et al.	151
2007	Person Authentication Using Brainwaves (EEG) and Maximum A Posteriori Model Adaptation [9]	Marcel and R. Millan	397
2009	Combined Neural Network Model Employing Wavelet Coefficients for EEG Signals Classification [10]	Ubeyli	260
2010	Expert model for detection of epileptic activity in EEG signature [11]	Gandhi et al.	177
2010	Real-Time EEG-Based Human Emotion Recognition and Visualization [12]	Liu et al.	343
2011	EEG signals classification using the K-means clustering and a multilayer perceptron neural network model [13]	Orhan et al.	592
2011	EEGLAB, SIFT, NFT, BCILAB, and ERICA: New Tools for Advanced EEG Processing [14]	Delorme et al.	480
2014	Real-Time Subject-Dependent EEG-Based Emotion Recognition Algorithm [15]	Liu and Sourina	107
2016	EEG-Based Emotion Recognition Approach for E-Healthcare Applications [16]	Ali et al.	207
2016	Why More is Better: Simultaneous Modeling of EEG, fMRI, and Behavioral Data [17]	Turner et al.	87
2021	A Novel Bi-hemispheric Discrepancy Model for EEG Emotion Recognition [18]	Li et al.	269
2022	EEG-Based Emotion Recognition: A Tutorial and Review [4]	Li et al.	36
2022	A large and rich EEG dataset for modeling human visual object recognition [19]	Gifford et al.	62

Table 14: Reproduced Studies

2.1.5.1. Preprocessing Options

The registry includes the following preprocessing variants:

Option	Definition
`clean`	Applies BioSemi 32 montage, notch filters at 50 Hz harmonics (50, 100, 150, 200, 250 Hz), a 4-45 Hz band-pass filter, and average reference. Epochs each trial from `tmin=-5 s` to `tmax=60 s`, resamples to `SFREQ_TARGET=128 Hz`, and reorders channels and trials into canonical Geneva and DEAP order.
`ica_clean`	Applies the same filtering and referencing as `clean`, then fits an ICA model with `n_components=EEG_ELECTRODES_NUM-1` using FastICA (`random_state=23`). Components are rejected based on EOG proxy channels (`Fp1`, `Fp2`) and high-variance sources exceeding the 99th percentile of component variance. The cleaned epochs are resampled to 128 Hz and reordered into canonical channel and trial order.
`unclean`	Applies no filtering or ICA. It epochs, resamples, and reorders the raw signals directly to provide an unfiltered baseline.

Table 15: Preprocessing registry

Filtering (band-pass / notch abstraction): models the linear time-invariant filtering used to attenuate line noise and restrict spectral content before feature extraction.

$$y[n] = \sum_{k=-K}^{K} h[k]x[n-k]$$

Average reference: removes common-mode activity by subtracting the instantaneous mean across channels, stabilizing downstream spectral and entropy features.

$$x_{c'}(t) = x_{c(t)} - \frac{1}{C} * \sum_{i=1}^{C} x_{i(t)}$$

ICA (linear mixing model): represents the observed EEG as a mixture of latent sources so artifact components can be identified and removed before segmentation.

$$x(t) = As(t), s(t) = Wx(t)$$

Additional preprocessing implementation details:

Aspect	Definition
Event extraction	Uses the stimulation channel and selects only event code `4`, with a subject-specific adjustment for subjects with incorrect event offsets.
Trial ordering	Derived from the DEAP ratings CSV.
Channel ordering	Derived from the DEAP channel mapping XLSX file.
Shape checks	Shape consistency across subjects is asserted; preprocessing aborts if any subject produces a different shape.
Artifacts	`params.json`, `metrics.json`, and `shape.csv` are written for both subject and trial stages via atomic directory writes.

Table 16: Preprocessing implementation details

2.1.5.2. Feature Extraction Options

Feature extraction is configured via the following option types:

Option Type	Definition
`ChannelPickOption(name, channel_pick)`	Specifies a subset of EEG channels; each name must belong to the canonical Geneva 32-channel list.
`FeatureOption(name, feature_channel_extraction_method)`	Wraps a per-segment extractor callable `(trial_data, channel_pick) -> np.ndarray`.
`SegmentationOption(time_window, time_step)`	Defines the sliding window. Constraints enforce `time_window > 0`, `time_step > 0`, `time_step <= time_window`, and `time_window <= BASELINE_SEC`. Names are deterministic as `w{time_window}_s{time_step}`.
`FeatureExtractionOption(preprocessing_option, channel_pick_option, feature_option, segmentation_option)`	Composes preprocessing, channel selection, feature extraction, and segmentation. The option name is the plus-joined string `preprocessing+feature+channel_pick+segmentation`, and the extractor is a partially applied callable that closes over the chosen channels.

Table 17: Feature extraction option types

Channel pick options define which electrodes are retained:

Option	Channels
minimal_frontal	["F3", "F4"]
minimal_frontal_parietal	["F3", "F4", "P3", "P4"]
minimal_frontopolar	["Fp1", "Fp2"]
frontal_prefrontal_4	["Fp1", "Fp2", "F3", "F4"]
emotiv_frontal_3	["AF3", "F4", "FC6"]
emotiv_frontal_4	["FC5", "F4", "F7", "AF3"]
minimal_temporal_augmented	["Fp1", "Fp2", "T7", "T8"]
balanced_classic_6	["Fp1", "Fp2", "F3", "F4", "P3", "P4"]
balanced_temporal	["F3", "F4", "T7", "T8", "P3", "P4"]
balanced_data_driven_5	["Fp1", "AF3", "FC2", "P3", "O1"]
optimized_gold_standard_8	["Fp1", "Fp2", "F3", "F4", "P3", "P4", "T7", "T8"]
optimized_lateral_mix	["F7", "F8", "T7", "T8", "P7", "P8", "Fz", "Cz"]
optimized_relief_nmi	["AF3", "AF4", "F3", "F4", "FC1", "FC2", "P3", "P4"]
posterior_parietal_14	["PO3", "PO4", "O1", "O2", "Oz", "P3", "P4", "P7", "P8", "Pz", "CP1", "CP2", "CP5", "CP6"]
frontal_full_10	["Fp1", "Fp2", "AF3", "AF4", "F3", "F4", "F7", "F8", "Fz", "FC1"]
standard_32	["Fp1", "AF3", "F3", "F7", "FC5", "FC1", "C3", "T7", "CP5", "CP1", "P3", "P7", "PO3", "O1", "Oz", "Pz", "Fp2", "AF4", "Fz", "F4", "F8", "FC6", "FC2", "Cz", "C4", "T8", "CP6", "CP2", "P4", "P8", "PO4", "O2"]

Table 18: Channel pick options

Segmentation options define window length and step size (in seconds), with names derived as `w{window}_s{step}`:

Option	time_window	time_step
w1.00_s0.25	1.0	0.25
w2.00_s0.25	2.0	0.25
w2.00_s2.00	2.0	2.0
w3.00_s0.15	3.0	0.15
w4.00_s4.00	4.0	4.0
w4.00_s1.00	4.0	1.0

Table 19: Segmentation options

Sliding-window segmentation: slices each trial into overlapping windows used for per-segment feature extraction and baseline/stimulus separation.

$$x_{m[n]} = x[n + m * S], 0 \leq n < L$$

Feature options define how each segment is transformed:

Option	Definition
`raw_waveform`	Returns zero-mean raw waveforms for the selected channels.
`psd`	Computes Welch log-power spectral density with `nperseg=min(SFREQ, n_samples)` and averages over bands `theta (4-7)`, `slow_alpha (8-10)`, `alpha (8-13)`, `beta (14-29)`, and `gamma (30-45)`; signals are scaled to microvolts before log transformation.
`de`	Computes differential entropy in bands `theta (4-8)`, `slow_alpha (8-10)`, `alpha (8-12)`, `beta (12-30)`, and `gamma (30-45)` using a 4th order Butterworth band-pass and the closed-form `0.5*log10(2*pi*e*std^2)` per channel.
`dasm`	Computes PSD-based left-right asymmetry features using predefined channel pairs (`Fp1-Fp2`, `AF3-AF4`, `F3-F4`, `F7-F8`, `FC5-FC6`, `FC1-FC2`, `C3-C4`, `T7-T8`, `CP5-CP6`, `CP1-CP2`, `P3-P4`, `P7-P8`, `PO3-PO4`, `O1-O2`). Each feature is the difference between left and right PSD bands.
`deasm`	Computes differential-entropy-based left-right asymmetry using the same channel pairs and differences.
`wavelet_energy_entropy_stats`	Applies a 5-level Daubechies-4 wavelet decomposition, maps coefficients to `delta`, `theta`, `alpha`, `beta`, `gamma` bands, and computes 31 features per channel (energies, energy ratios, log ratios, absolute log ratios, Shannon entropies, mean, standard deviation, and first and second difference statistics).
`higuchi_fd_frontal`	Computes Higuchi fractal dimension with `k_max=8` for each channel and adds an AF3-F4 asymmetry term when both channels are present.
`hoc_stat_fd_frontal4`	Computes 36 higher-order crossing counts, 6 statistical features, and Higuchi fractal dimension per channel, then applies z-score normalization across channels. This feature set is designed for the four-electrode montage `["FC5", "F4", "F7", "AF3"]` but can be applied to any configured channel pick.

Table 20: Feature options

Zero-mean waveform: removes the segment-wise DC offset to emphasize oscillatory structure in `raw_waveform` features.

$$x_0[n] = x[n] - \frac{1}{N} * \sum_{i=1}^{N} x[i]$$

Welch PSD: estimates band power used by `psd` and the left-right differences in `dasm`.

$$P_{xx}(f) = \frac{1}{M} * \sum_{m=1}^{M} | \sum_{n=0}^{L-1} w[n] x_{m[n]} e^{-i*2*\pi*f*n} \ |^2$$

Differential entropy: summarizes band-limited signal complexity in `de` and its asymmetry variant `deasm`.

$$H_d = \frac{1}{2} * \log_{10(2*\pi*e*\sigma^2)}$$

Left-right asymmetry (DASM/DEASM): quantifies hemispheric imbalance by subtracting homologous channel features within each band.

$$A_b = F_L, b - F_R, b$$

Wavelet energy and entropy: captures the time-frequency energy distribution used in `wavelet_energy_entropy_stats`.

$$E_b = \sum_n |c_{b[n]}|^2, p_b = \frac{E_b}{\sum_k} E_k, H = -\sum_k p_k * \log(p_k)$$

Higuchi fractal dimension: measures fractal complexity used in `higuchi_fd_frontal` and included in `hoc_stat_fd_frontal4`.

$$\log L(k) = a - D * \log k$$

Higher-order crossings: count sign changes around threshold r to capture nonlinear dynamics in `hoc_stat_fd_frontal4`.

$$C_r = \sum_{n=2}^{N} I((x[n] - r) * (x[n-1] - r) < 0)$$

Extraction behavior:

Aspect	Definition
Baseline window	For each trial, the baseline feature is computed using the most recent window that f its within the 5 s baseline period. All subsequent windows are extracted from the st imulus portion using the configured sliding window.
Feature storage	Features are stored as a DataFrame with one row per segment and trial labels retain ed; baseline features are stored under `metadata/baseline/`.
Shape checks	Shape consistency is verified across trials. If segmentation or feature extraction yiel ds inconsistent shapes, the extractor raises an error rather than emitting partial outp uts.

Table 21: Extraction behavior

2.1.5.3. Model Training Options

Dataset construction and training orchestration are encoded by:

Option Type	Definition
`BuildDatasetOption(target, random_seed, use_size, test_size, target_kind, feature_scaler)`	Describes the label target, split sizes, task kind, and scaling strategy, with deterministic names of the form `target+use{use_size}+test{test_size}+seed{random_seed}+{target_kind}+{feature_scaler}`.
`TrainingDataOption(feature_extraction_option, build_dataset_option)`	Loads feature frames, encodes targets, applies scaling, and generates deterministic train/test splits.
`TrainingMethodOption(name, target_kind, backend, epochs, batch_size, optimizer_builder, criterion_builder, num_workers, pin_memory, drop_last, device, batch_formatter, train_epoch_fn, evaluate_epoch_fn, prediction_collector, fit_kwargs)`	Defines the torch or sklearn training procedure and required callables.
`TrainingOption(training_data_option, training_method_option)`	Validates target-kind compatibility and constructs dataloaders for torch backends.
`ModelOption(name, model_builder, target_kind, backend, output_size)`	Describes the model architecture or estimator; `ModelTrainingOption(model_option, training_option)` resolves output sizes from class labels and defines run paths.

Table 22: Model training option types

Classification target variants are treated as mutually compatible, allowing `classification`, `classification_5`, and `classification_3` datasets to share model and training method implementations. Shape uniformity and target-kind compatibility are enforced before training begins.

Dataset building is controlled by `BuildDatasetOption` and `TrainingDataOption`:

Rule	Definition
Registry enumeration	Enumerates Cartesian products of `target in {valence, arousal}`, `use_size in {1.0, 0.3}`, `test_size in {0.2, 0.3}`, `target_kind in {classification, classification_5, classification_3, regression}`, `feature_scaler in {standard, minmax}`, with `random_seed=42`.
Classification bins	`classification` keeps 9-point labels; `classification_5` maps to five bins (1-2, 3-4, 5, 6-7, 8-9); `classification_3` maps to three bins (1-3, 4-6, 7-9). Regression preserves continuous ratings.
Label encoding	For classification, labels are encoded as integer indices and the canonical class label set is stored for later decoding.
Feature scaling	`feature_scaler=standard` and `feature_scaler=minmax` are applied per segment by flattening each segment, fitting a scaler on the training set, transforming, and reshaping back to the original segment shape. `feature_scaler=none` is supported by the code path but not used in the registry.
Split strategy	Splits are generated by shuffling indices with `random_seed`, selecting the first `use_size` fraction of segments, and allocating `round(test_size * used)` segments to the test set (at least one). Remaining segments form the training set.

Table 23: Dataset construction rules

Training methods define optimization and data-loading details:

Training method	Definition
`adam_regression`	Torch backend with `epochs=10`, `batch_size=64`, `optimizer=Adam(lr=1e-3, weight_decay=1e-5)`, `criterion=MSELoss`, `device=cpu`, and the same Conv1d-compatible helpers.
`adam_classification`	Torch backend with `epochs=30`, `batch_size=256`, `optimizer=Adam(lr=1e-3, weight_decay=1e-5)`, `criterion=CrossEntropyLoss`, `device=cpu`, and Conv1d-compatible batch formatting and train/eval loops.
`sgd_bihdm_classification`	Torch backend with `epochs=30`, `batch_size=200`, `optimizer=SGD(lr=3e-3, momentum=0.9, weight_decay=0.95)`, `criterion=CrossEntropyLoss`, `device=cpu`, and Conv1d-compatible helpers aligned with BiHDM training.
`sklearn_default_classification`	Sklearn backend placeholder for classification targets.
`sklearn_default_regression`	Sklearn backend placeholder for regression targets.

Table 24: Training method options

Mean squared error (regression): objective used by regression runs to penalize large prediction errors on continuous ratings.

$$L_{\text{mse}} = \frac{1}{N} * \sum_{i=1}^{N} (y_i - y_i^p)^2$$

Softmax cross-entropy (classification): objective used by classification runs to align logits with discrete emotion bins.

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{K} \exp(z_{ij})}, L_{\text{ce}} = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} * \log(p_{ik})$$

Gradient descent update: generic optimizer step for torch training loops in `adam_*` and `sgd_*` methods.

$$\theta_{\{t+1\}} = \theta_t - \eta * g_t$$

Model options include both torch architectures and sklearn estimators:

Model option	Definition
`cnn1d_n1_classification`, `cnn1d_n1_regression`	Three-block 1D CNN (Conv1d 128-128-64 with BatchNorm and MaxPool) followed by a fully connected head using lazy input sizing, dropout (0.2), and tanh/ReLU activations. Output size is 9 for classification and 1 for regression.
`cfcnn_classification`	Channel-frequency 2D CNN that reshapes band features into a `(channels x bands)` map, applies two Conv2d layers (16/32) with BatchNorm, then a lazy fully connected layer (128), dropout (0.3), and classification head.
`combined_wavelet_mlp_classification`	Two-stage ensemble: three first-level MLP branches (hidden size 20) feed a second-level MLP (hidden size 25) with sigmoid activations to combine wavelet features.
`kmeans_wavelet_mlp_classification`	Single-hidden-layer MLP (hidden size 5) with sigmoid activation, designed for K-means wavelet probability features.
`bihdm_classification`	Bi-hemispheric discrepancy model with GRU streams per hemisphere, discrepancy features, high-level GRUs, and a linear classifier head (feature_dim=5, dl=32, dg=32, do=16).
`mlp_classification`	Two-hidden-layer MLP with lazy input sizing, ReLU activations, and dropout rates 0.2 and 0.1; output size is 9.

Table 25: Model options (torch)

Feedforward layer: core transformation in MLP and CNN heads, mapping extracted features to logits or regression outputs.

$$h^l = \varphi\big(W^l h^{l-1} + b^l\big)$$

1D convolution: captures local temporal patterns in segment features for `cnn1d_*` architectures.

$$y_{j[t]} = \sum_i \sum_{k=0}^{K-1} w_{\{j,i,k\}} * x_{i[t-k]}$$

Model option	Definition
`logreg_sklearn`	LogisticRegression with `max_iter=500`, `n_jobs=-1`.
`rf_regression_sklearn`, `rf_classification_sklearn`	RandomForest with `n_estimators=200`, `random_state=23`, `n_jobs=-1`, and `class_weight=balanced` for classification.
`svc_rbf_sklearn`	SVC with `kernel=rbf`, `gamma=scale`, `C=10.0`, `probability=True`.
`linear_svc_sklearn`	LinearSVC with `C=1.0`, `class_weight=balanced`.
`svr_rbf_sklearn`	SVR with `kernel=rbf`, `gamma=scale`, `C=10.0`, `epsilon=0.1`.
`gb_regression_sklearn`	GradientBoostingRegressor with `random_state=23`.
`sgd_classifier_sklearn`	SGDClassifier with `loss=log_loss`, `penalty=l2`, `max_iter=1000`, `tol=1e-3`, `n_jobs=-1`.
`ridge_regression_sklearn`	Ridge with `alpha=1.0`, `random_state=23`.
`linear_regression_sklearn`	LinearRegression with `n_jobs=-1`.
`elasticnet_regression_sklearn`	ElasticNet with `alpha=0.001`, `l1_ratio=0.5`, `random_state=23`.
`knn_classifier_sklearn`	KNeighborsClassifier with `n_neighbors=5`, `weights=distance`, `metric=euclidean`, `n_jobs=-1`.
`qda_classifier_sklearn`	QuadraticDiscriminantAnalysis with `reg_param=0.01`.

Table 26: Model options (sklearn)

Logistic regression: linear baseline classifier used in `logreg_sklearn` for separable feature spaces.

$$p(y = 1 \mid x) = \frac{1}{1 + \exp(-(w^T x + b))}$$

RBF SVM: nonlinear classifier in `svc_rbf_sklearn` using a Gaussian kernel to separate complex decision boundaries.

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b, K(x, x') = \exp(-\gamma * |x - x'|^2)$$

ElasticNet: regularized regression in `elasticnet_regression_sklearn` balancing sparsity and shrinkage.

$$w^* = \min_w \|y - Xw\|_2^2 + \alpha * (\lambda * \|w\|_1 + (1 - \lambda) * \|w\|_2^2)$$

KNN classifier: nonparametric classifier in `knn_classifier_sklearn` based on local neighborhood votes.

$$y^p = \text{argmax}_c \sum_{i \in N_{k(x)}} I(y_i = c)$$

QDA: generative classifier in `qda_classifier_sklearn` assuming class-conditional Gaussian densities.

$$\delta_{k(x)} = -\frac{1}{2} * \log|\Sigma_k| - \frac{1}{2} * (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) + \log \pi_k$$

Random forest: ensemble model in `rf_*_sklearn` averaging tree predictions to reduce variance.

$$f(x) = \frac{1}{T} * \sum_{t=1}^{T} f_{t(x)}$$

Training execution and metrics:

Aspect	Definition
Torch training	Seeds PyTorch for determinism, selects `cpu` or `cuda` devices based on availabilit y, and tracks epoch-level `train_loss`, `train_mae`, `val_loss`, and `val_mae`. The model checkpoint with the lowest validation loss is retained.
Classification MAE	Computed by mapping predicted class indices back to class center values before computing absolute error.
Sklearn training	Fits once on the flattened arrays, predicts on train and test splits, and reports the same metrics format as torch runs.
Metrics	Regression metrics include MAE, median absolute error, MAPE, MSE, RMSE, R2, explained variance, and max error. Classification metrics include accuracy, b alanced accuracy, precision, recall, and F1 (macro, micro, weighted), and the co nfusion matrix.

Table 27: Training execution and metrics

Mean absolute error: primary regression error metric reported for continuous targets.

$$\text{MAE} = \frac{1}{N} * \sum_{i=1}^{N} |y_i - y_i^p|$$

Root mean squared error: emphasizes larger regression errors in reporting.

$$\text{RMSE} = \sqrt{\frac{1}{N} * \sum_{i=1}^{N} \left(y_i - y_i^p\right)^2}$$

Coefficient of determination: summarizes explained variance in regression results.

$$R^2 = 1 - \frac{\sum_{i=1}^{N} \left(y_i - y_i^p\right)^2}{\sum_{i=1}^{N} \left(y_i - y_m\right)^2}$$

F1 score: balances precision and recall for classification performance comparisons.

$$\text{F1} = \frac{2 * P * R}{P + R}$$

Accuracy: overall classification correctness reported alongside balanced metrics.

$$\text{Acc} = \frac{1}{N} * \sum_{i=1}^{N} I(y_i = y_i^p)$$

2.1.5.4. Artifact Layout

Stage	Location	Notes
Preprocessing (subject)	`data/DEAP/generated/<root_dir>/subject`	`metadata/` includes `params.json`, `metrics.json`, and `shape.csv`.
Preprocessing (trial)	`data/DEAP/generated/<root_dir>/trial`	`metadata/` includes `params.json`, `metrics.json`, and `shape.csv`.
Feature extraction	`data/DEAP/generated/<root_dir>/feature/<pre+feat+ch+seg>/`	Baseline arrays are stored under `metadata/baseline/`; extraction metadata is stored under `metadata/params.json`, `metadata/metrics.json`, and `metadata/shape.csv`.
Training runs	`results/<target>/<target_kind>/<timestamp>/`	`params.json`, `params.sha256`, `metrics.json`, `splits.json`, and the serialized model artifact (`best_model.pt` for torch, `best_model.joblib` for sklearn).
Atomic writes	All stages	Atomic directory writes prevent partial or corrupted outputs and parameter hashes detect duplicate runs.

Table 28: Artifact layout

2.2. Application Development

Having established the pipeline and evaluation artifacts, we now describe how the selected configurations are embedded in a real-time application workflow. This step bridges the systematic evaluation in the pipeline to the deployment theme, demonstrating how the identified robust stacks can be deployed under the real-world constraints outlined in the Introduction.

2.2.1. Design

The workflow of the application is shown in Figure 2. A device reads electromagnetic signals from the user's scalp using EEG electrodes and streams them to the classification algorithm built with the selected techniques. The device then streams the classification results to the client app on the user's mobile phone, which gives the user real-time feedback about their emotions.

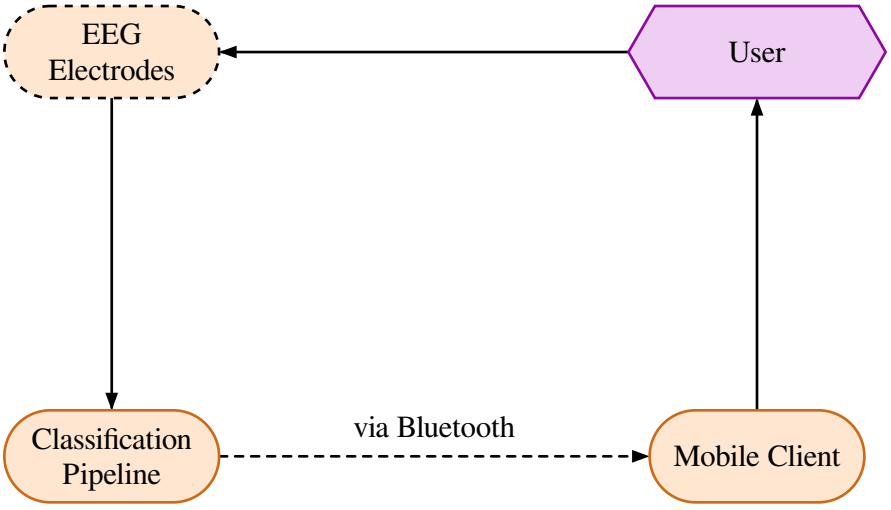

Figure 2: Real-time EEG Emotion Application Workflow

Due to limited hardware support (i.e., unavailable EEG and Bluetooth devices), mock endpoints are used for EEG signal collection and classification-result streaming instead of physical hardware.

2.2.2. Real-time Emotion Recognition

The selected pipeline configurations can be deployed on a device by exporting the preprocessing parameters, feature scaler, and trained model weights as serialized artifacts. On-device inference follows the same deterministic stages: the device buffers incoming EEG samples, applies the chosen preprocessing (e.g., ICA-cleaned or clean bandpass), segments the stream into sliding windows, extracts features (DE or PSD), and normalizes them with the fitted scaler. Each window yields a valence or arousal prediction from the trained classifier, producing a continuous stream of emotion estimates at the window rate.

To meet real-time requirements, the inference loop operates in a streaming fashion with small, overlapping windows (e.g., 2–3 s windows with sub-second steps), ensuring low latency while preserving enough temporal context for stable classification. For low-channel devices, the recommended configurations from the Results section reduce computation by limiting channels and using efficient features, which allows the model to run on embedded hardware without a GPU.

Predictions are packaged with timestamps and optional confidence metrics (e.g., softmax scores or class probabilities) and sent to the mobile client through a lightweight streaming interface. In the prototype, this is implemented with mock endpoints that emulate Bluetooth transport; in a deployed system, the same payloads can be transmitted over BLE or a local socket to the client, which renders the live emotional state and stores it for user feedback.

The application component is presented as a functional prototype that validates end-to-end integration of sensing, inference, and visualization. Quantitative evaluation in this study focuses on pipeline performance; usability testing and latency measurements on physical hardware are left for future work.

2.2.3. Mobile Client Application

The mobile client is implemented as an Expo-based React Native app written in TypeScript. It uses React Native core components with custom neumorphic UI components, `react-native-svg` for charts, `@react-native-async-storage/async-storage` for local persistence, and Expo modules (`expo-file-system`, `expo-sharing`, `expo-document-picker`) for data import/export. Icons are provided by `lucide-react-native`, and the mock Bluetooth stream is simulated in-app for the prototype. In the current build, the Bluetooth link is emulated with a mock device stream; real BLE transport can replace the mock interface in deployment.

Major features include:

1. Real-time monitoring with a live connection state, affect map and trend views, and valence/arousal readouts.
2. History analytics with time-scale filters, averages, range analysis, emotion distribution, and activity heatmaps.
3. Settings for theme and language (English/Chinese), data management (view, import, export, clear), and developer tools for mock streaming and mock data generation.

2.2.3.1. UI Design

Neomorphic elements were chosen for a modern, intuitive interface. Soft shadows and rounded cards emphasize key metrics, while consistent spacing and typography keep the screen readable at a glance. The accent palette differentiates valence and arousal trends and reinforces status cues without overwhelming the content.

All screenshots below are captured using the iPhone 13 mini layout.

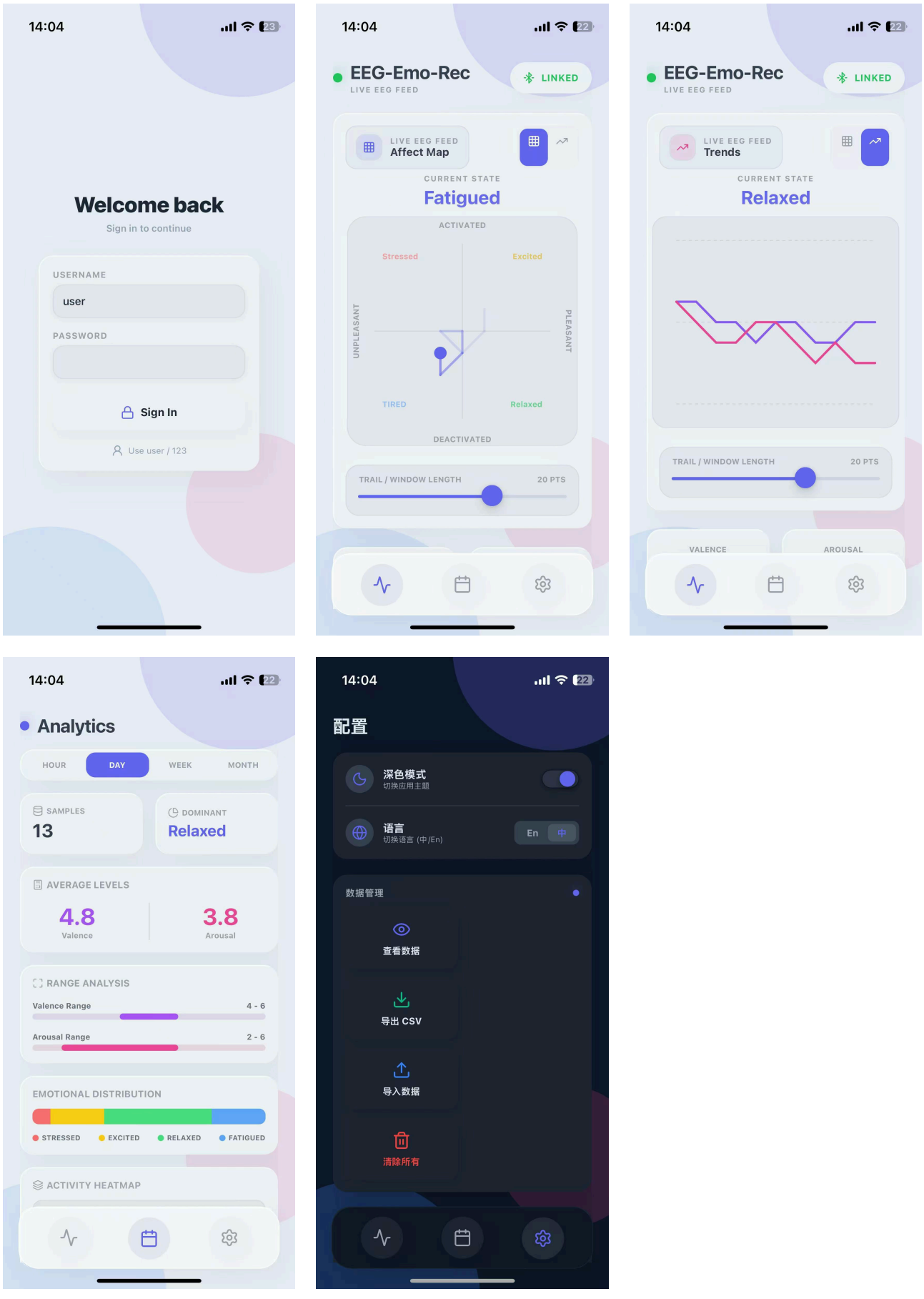

Figure 3: Mobile client screens: authentication, real-time monitoring (two views), analytics, and settings.

3. Results

With the methodology defined, we report the quantitative performance of the pipeline configurations, directly addressing the goal of identifying robust technique stacks for deployment under real-world constraints. Pipeline configurations that reproduce the setups of the chosen studies (with extended option sets) are run, along with an additional set of randomly combined configurations, producing 1176 result artifacts in total. The Results section therefore concentrates on quantitative pipeline performance; the application prototype is validated through functional end-to-end integration rather than formal usability or latency evaluation.

In the following sections, classification performances are summarized with test macro-F1 and balanced accuracy to reflect class imbalance; regression performances are summarized with RMSE and R2.

3.1. Option-Level Performance

To expose which stages of the pipeline contribute the most to downstream accuracy, we aggregate runs by option group. For each target-kind, we take the best test macro-F1 for valence and arousal separately, then average those two maxima per option. Figure 4, Figure 5, and Figure 6 report the top-performing options within each group. These breakdowns highlight how individual components interact to support the overall deployment theme, with noise-handling preprocessing and efficient features emerging as key enablers for constrained environments.

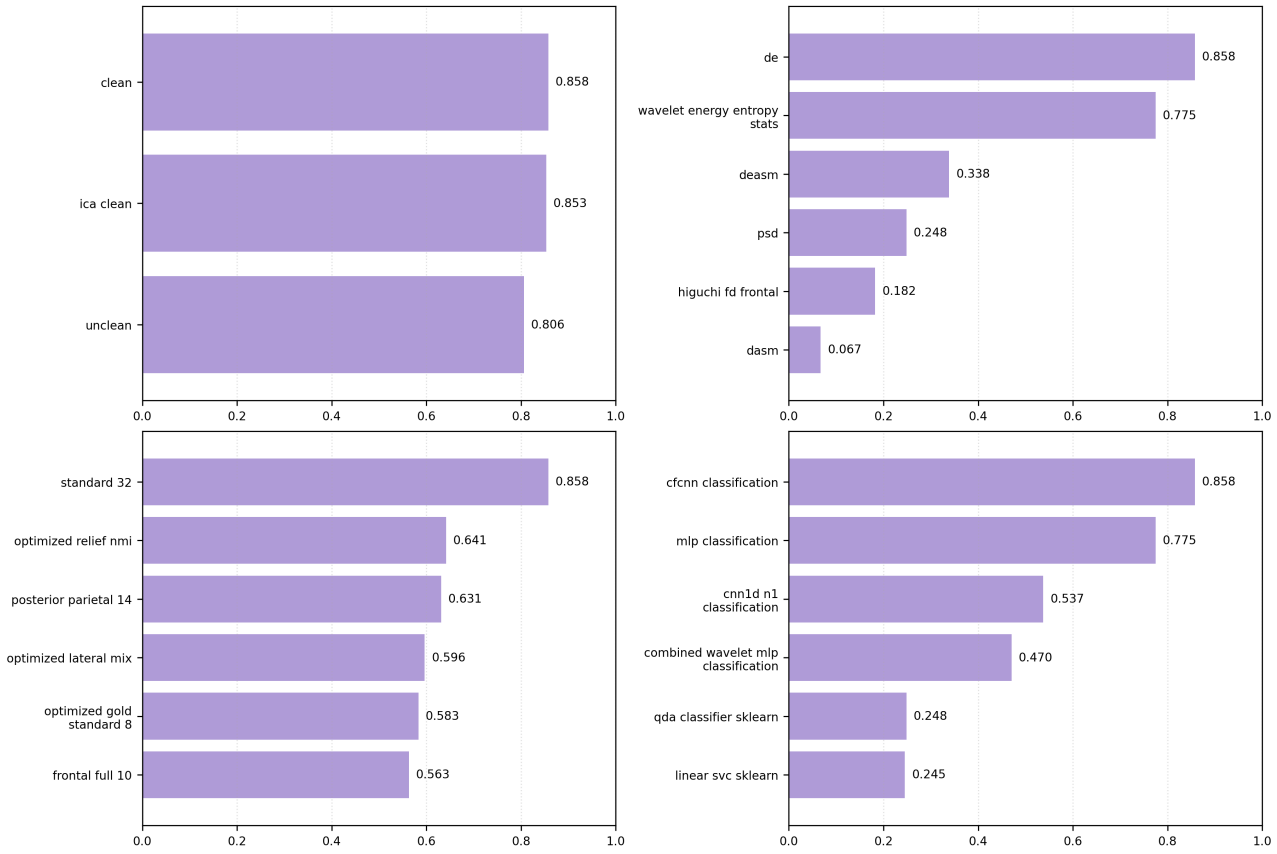

Figure 4: Option-level best test macro-F1 for `classification` (9 classes). Each bar is the mean of the best valence and arousal runs for the same option.

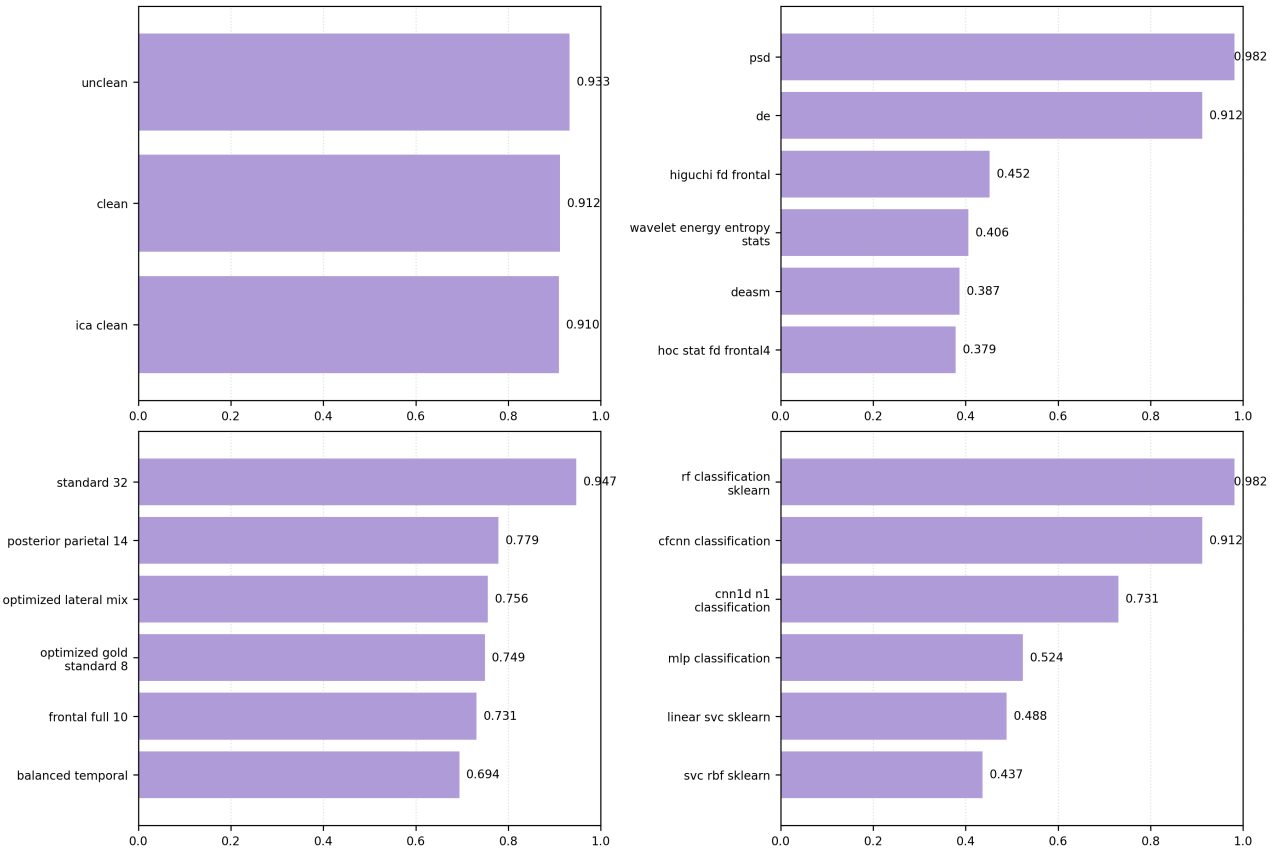

Figure 5: Option-level best test macro-F1 for `classification_3`. Each bar is the mean of the best valence and arousal runs for the same option.

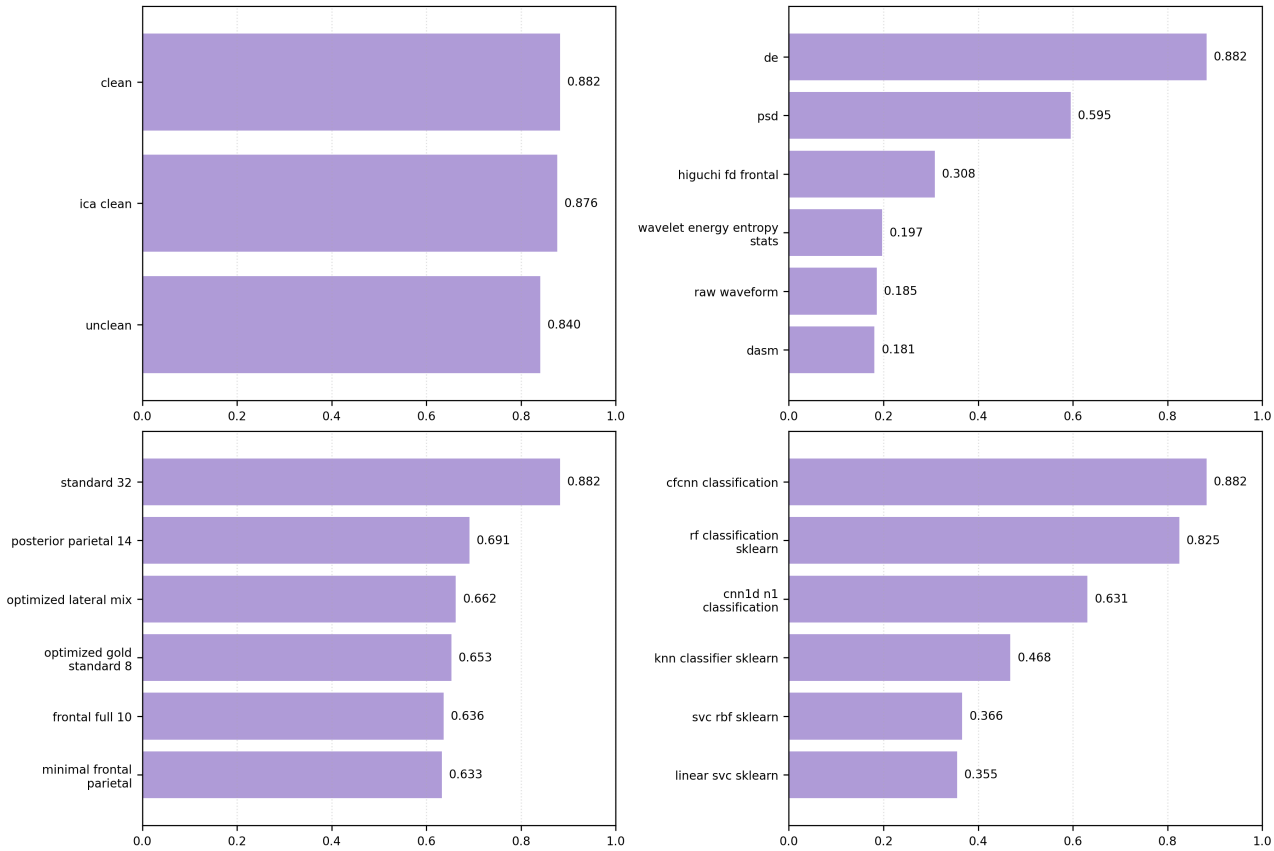

Figure 6: Option-level best test macro-F1 for `classification_5`. Each bar is the mean of the best valence and arousal runs for the same option.

The option-level trends show that 9-class and 5-class settings favor clean preprocessing with DE features and the CF-CNN classifier, while 3-class results benefit from PSD features paired with a tree-based model. Across all target kinds, the standard 32-channel montage remains the best-performing channel pick for maximum accuracy.

3.2. Best-Performing Configurations

The strongest overall 9-class and 5-class pipelines are consistent across valence and arousal: clean preprocessing, DE features, the full 32-channel montage, and the CF-CNN classifier. Table 29, Table 30, and Table 31 list the exact parameters for the highest-scoring configurations. These top performers serve as benchmarks for the low-channel analysis that follows, aligning with the study's focus on practical, constrained deployment.

Parameter	Valence (best)	Arousal (best)
Test macro-F1	0.857	0.858
Test balanced accuracy	0.857	0.855
Test accuracy	0.855	0.859
Preprocessing	clean	clean
Feature	de	de
Channel pick	standard_32	standard_32
Segmentation	w2.00_s0.25	w2.00_s0.25
Model	cfcnn_classification	cfcnn_classification
Training method	adam_classification	adam_classification
Feature scaler	standard	standard
Use size	1.0	1.0
Test size	0.3	0.3

Table 29: Best `classification` (9-class) pipeline configurations.

Parameter	Valence (best)	Arousal (best)
Test macro-F1	0.876	0.889
Test balanced accuracy	0.872	0.885
Test accuracy	0.876	0.891
Preprocessing	clean	clean
Feature	de	de
Channel pick	standard_32	standard_32
Segmentation	w2.00_s0.25	w2.00_s0.25
Model	cfcnn_classification	cfcnn_classification
Training method	adam_classification	adam_classification
Feature scaler	standard	standard
Use size	1.0	1.0
Test size	0.3	0.3

Table 30: Best `classification_5` pipeline configurations.

Parameter	Valence (best)	Arousal (best)
Test macro-F1	0.982	0.912
Test balanced accuracy	0.978	0.909
Test accuracy	0.983	0.916
Preprocessing	unclean	clean
Feature	psd	de
Channel pick	standard_32	standard_32
Segmentation	w3.00_s0.15	w2.00_s0.25
Model	rf_classification_sklearn	cfcnn_classification
Training method	sklearn_default_classification	adam_classification
Feature scaler	minmax	standard
Use size	1.0	1.0
Test size	0.2	0.3

Table 31: Best `classification_3` pipeline configurations.

Regression runs remain substantially weaker: the best RMSE values (around 1.92–2.05) correspond to R2 below 0.10, so classification targets are preferred for practical deployment.

3.3. Low-Channel and Noisy-Data Deployment

Low-channel performance is evaluated at three operating points: <= 4, <= 8, and <= 12 channels. For each scale, we report the top channel picks by averaging the best macro-F1 values for valence and arousal. Figure 7, Figure 8, and Figure 9 summarize these rankings for the 3-class and 5-class settings. This analysis directly ties back to the Introduction's emphasis on few-channel constraints, identifying deployable configurations for consumer devices.

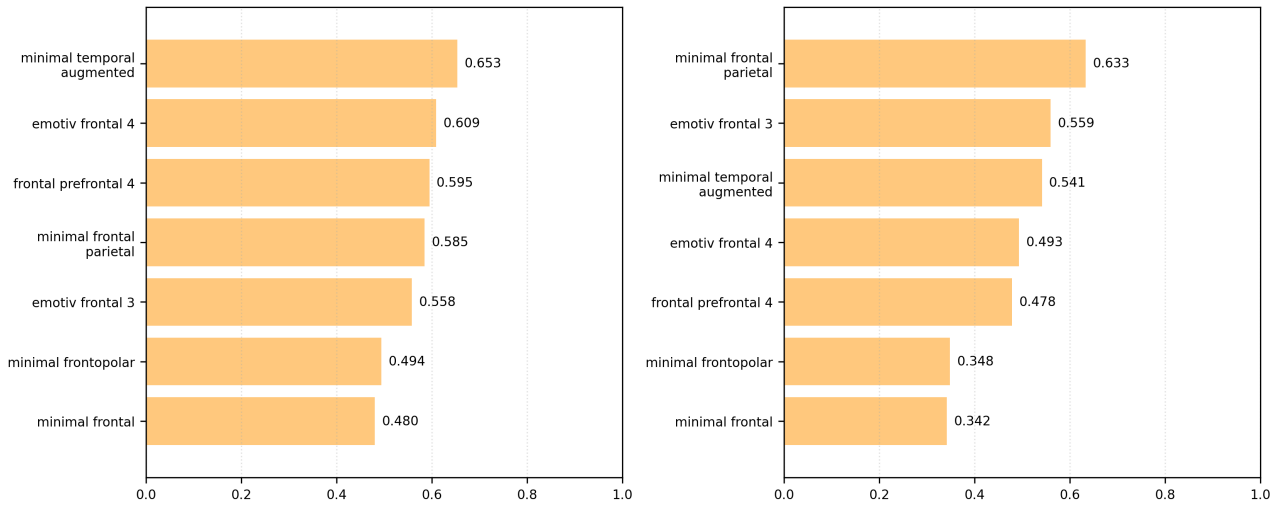

Figure 7: Low-channel (<= 4 channels) option performance. Bars report the mean of best macro-F1 per target, highlighting which compact channel sets are most competitive.

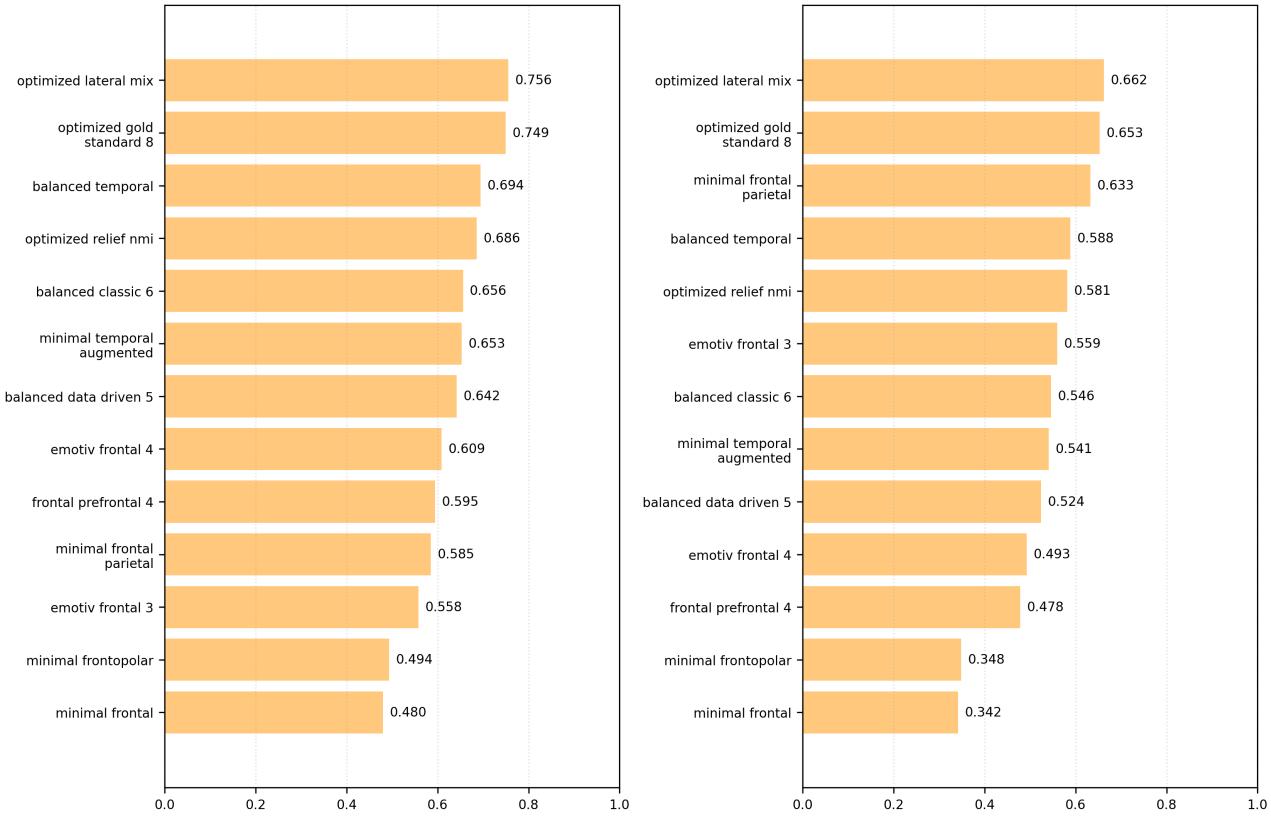

Figure 8: Low-channel (<= 8 channels) option performance.

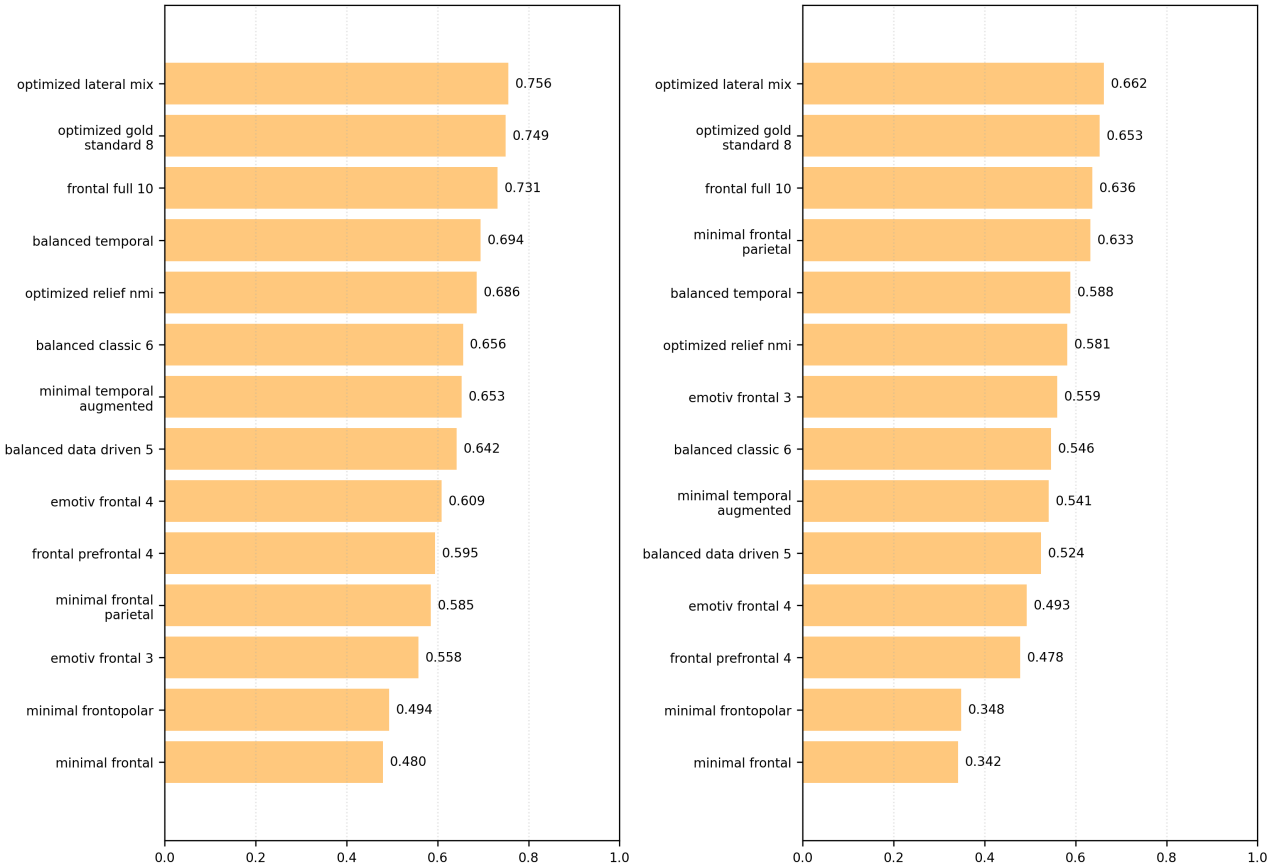

Figure 9: Low-channel (<= 12 channels) option performance.

Target kind	<= 4 channels	<= 8 channels	<= 12 channels
classification (9)	minimal_temporal_augmented (0.464)	optimized_relief_nmi (0.641)	optimized_relief_nmi (0.641)
classification_3	minimal_temporal_augmented (0.653)	optimized_lateral_mix (0.756)	optimized_lateral_mix (0.756)
classification_5	minimal_frontal_parietal (0.633)	optimized_lateral_mix (0.662)	optimized_lateral_mix (0.662)

Table 32: Low-channel summary: best channel pick and mean macro-F1 (valence/arousal averaged).

For the target application (few channels, noisy signals), the most competitive and consistent pipeline at the <= 4-channel scale is the `minimal_temporal_augmented` montage with ICA cleaning, DE features, and the CF-CNN classifier. This configuration is shared by the best low-channel valence and arousal runs, trading absolute accuracy for robustness and compact sensing. Table 33 summarizes this recommended stack.

Parameter	Valence	Arousal
Test macro-F1	0.641	0.666
Test balanced accuracy	0.633	0.657
Test accuracy	0.663	0.686
Preprocessing	ica_clean	ica_clean
Feature	de	de
Channel pick	minimal_temporal_augmented	minimal_temporal_augmented
Segmentation	w2.00_s0.25	w2.00_s0.25
Model	cfcnn_classification	cfcnn_classification
Training method	adam_classification	adam_classification
Feature scaler	standard	standard
Use size	1.0	1.0
Test size	0.3	0.3

Table 33: Recommended low-channel pipeline (`classification_3`, <= 4 channels).

Parameter	Valence	Arousal
Test macro-F1	0.753	0.758
Test balanced accuracy	0.745	0.750
Test accuracy	0.767	0.769
Preprocessing	clean	clean
Feature	de	de
Channel pick	optimized_lateral_mix	optimized_lateral_mix
Segmentation	w2.00_s0.25	w2.00_s0.25
Model	cfcnn_classification	cfcnn_classification
Training method	adam_classification	adam_classification
Feature scaler	standard	standard
Use size	1.0	1.0
Test size	0.3	0.3

Table 34: Best low-channel pipeline (`classification_3`, <= 8 and <= 12 channels).

At <= 8 and <= 12 channels, the 3-class setting converges to the same 8-channel `optimized_lateral_mix` montage with clean preprocessing and DE features, indicating that adding channels beyond eight does not change the optimal configuration within this search space. The 5-class setting also prefers `optimized_lateral_mix` at these scales, but with lower macro-F1 values (0.66). The 9-class setting is markedly less stable under low-channel constraints, dropping to 0.46 macro-F1 at <= 4 channels and exhibiting target-specific channel/model choices at <= 8 channels. A single shared pipeline for both targets therefore favors the ICA + DE + CF-CNN stack at <= 4 channels, and the clean + DE + CF-CNN stack with `optimized_lateral_mix` at <= 8/12 channels, offering the best stability under low-channel and noisy conditions.

4. Discussion

This section interprets the results in the context of deployment constraints and connects the modeling choices to the application scenario; limitations and future directions follow. We find that many pipeline configurations underperform the results reported in the original studies, suggesting limited generalization when evaluated on (i) datasets different from those they were trained on and (ii) partially modified configurations .

The strongest configurations for 9-class and 5-class classification converge on clean preprocessing, DE features, and the CF-CNN classifier, suggesting that these combinations are the most transferable across targets when the label space is dense. The 3-class setting, by contrast, benefits from PSD features with a tree-based model, which likely reflects the coarser decision boundaries and stronger separation in lower-frequency band power features. Regression results remain weak, with low R2 even at best RMSE, indicating that direct continuous prediction is not yet competitive under the current pipeline choices and data constraints. These patterns align with the Introduction's emphasis on practical deployment, where discrete classification offers more reliable performance in noisy, low-compute settings.

Channel count is a dominant driver of performance. Full 32-channel montages consistently outperform reduced selections, but the low-channel analysis reveals two practical operating points: at <= 4 channels, `minimal_temporal_augmented` with ICA + DE + CF-CNN provides the most stable tradeoff between noise tolerance and accuracy; at <= 8 and <= 12 channels, `optimized_lateral_mix` with clean preprocessing and DE features becomes the best shared option for the 3-class setting. The low-channel setting shows the steepest

degradation in low-channel conditions, which is consistent with the higher class complexity and increased sensitivity to spatial coverage.

Taken together, the results indicate that low-channel deployment should favor robust preprocessing (ICA when noisy) and compact, information-dense channel sets rather than solely relying on more complex models. For practical devices, the recommended stacks from the Results section provide a defensible balance between accuracy, channel budget, and expected noise conditions, directly supporting the prototype application's real-time workflow.

The application prototype connects these configurations to a realistic usage flow and frames the study as a step toward real-world deployment, but its evaluation is limited to functional validation on mock streams. As a result, user experience and on-device latency remain open considerations that should be addressed alongside model performance in future studies.

5. Limitations and Future Directions

Several limitations exist for this pipeline:

- Only a single data source (DEAP) is used, limiting the study's generalizability across datasets.
- The number of reproduced studies is limited. Many studies also share overlapping techniques, reducing the variety of methods.
- Computational resource consumption is approximated by runtime rather than direct CPU or battery measurements, which might not fully represent computational efficiency.
- The prototype application is not evaluated with formal usability studies or measured latency on physical hardware; integration is validated via mock streams only.
- Due to time and computational resource constraints, a full-grid search or a finer-grained random search over all possible configurations was not conducted, so better configurations may have been missed.

Given these limitations, future studies should pursue the following directions:

- Include more datasets, primarily large, public ones such as SEED and DREAMER [20], [21], [22]
- Add more published studies to the reproduction list
- Run a finer-grained search of the pipeline configurations, potentially incorporating hyperparameter optimization techniques like genetic algorithms.

As the pipeline was designed and built with modularity and extensibility in mind, future improvements can be made to help overcome many of the current limitations.

For the mobile client, the current prototype does not support a functional endpoint with a true Bluetooth connection to the EEG device and relies on mock streaming interfaces.

6. Conclusion

In summary, this work presents a reproducible EEG emotion-recognition pipeline and a prototypical application workflow tailored to real-world constraints, positioning application development as a core theme alongside model evaluation. Across 1176 configurations on DEAP, clean preprocessing with DE features and a CF-CNN classifier yields the strongest 9- and 5-class performance, while PSD with tree-based models is favored in the 3-class setting. Regression remains unreliable under the current setup, reinforcing the practicality of discrete classification targets.

The low-channel analysis identifies stable, application-ready options: a 4-channel temporal montage with ICA + DE + CF-CNN for noisy, minimal-sensor scenarios, and an 8-channel lateral montage with clean preprocessing and DE features for higher-capacity devices. These results provide concrete guidance for deploying EEG emotion recognition on consumer-grade hardware and establish a foundation for future extensions, including broader datasets, finer hyper-parameter searches, and hardware-integrated validation.

By linking systematic pipeline evaluation to a functional prototype, this study advances toward the real-world deployment goals outlined in the Introduction.

References

[1] "Emotion Recognition," *Wikipedia*, Jun. 2025.

[2] S. K. Khare, V. Blanes-Vidal, E. S. Nadimi, and U. R. Acharya, "Emotion Recognition and Artificial Intelligence: A Systematic Review (2014–2023) and Research Recommendations," *Information Fusion*, vol. 102, p. 102019, Feb. 2024, doi: 10.1016/j.inffus.2023.102019.

[3] M. Jafari *et al.*, "Emotion Recognition in EEG Signals Using Deep Learning Methods: A Review," *Computers in Biology and Medicine*, vol. 165, p. 107450, Oct. 2023, doi: 10.1016/j.compbiomed.2023.107450.

[4] X. Li *et al.*, "EEG Based Emotion Recognition: A Tutorial and Review," *ACM Comput. Surv.*, vol. 55, no. 4, pp. 1–57, Nov. 2022, doi: 10.1145/3524499.

[5] D. Dadebayev, W. W. Goh, and E. X. Tan, "EEG-based Emotion Recognition: Review of Commercial EEG Devices and Machine Learning Techniques," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 7, pp. 4385–4401, Jul. 2022, doi: 10.1016/j.jksuci.2021.03.009.

[6] S. Koelstra *et al.*, "DEAP: A Database for Emotion Analysis ;Using Physiological Signals," *IEEE Trans. Affective Comput.*, vol. 3, no. 1, pp. 18–31, Jan. 2012, doi: 10.1109/t-affc.2011.15.

[7] O. David and K. J. Friston, "A Neural Mass Model for MEG/EEG:: Coupling and Neuronal Dynamics," *NeuroImage*, vol. 20, no. 3, pp. 1743–1755, Nov. 2003, doi: 10.1016/j.neuroimage.2003.07.015.

[8] F. Wendling, A. Hernandez, J.-J. Bellanger, P. Chauvel, and F. Bartolomei, "Interictal to Ictal Transition in Human Temporal Lobe Epilepsy: Insights from a Computational Model of Intracerebral EEG," *J Clin Neurophysiol*, vol. 22, no. 5, pp. 343–356, Oct. 2005.

[9] S. Marcel and J. D. R. Millan, "Person Authentication Using Brainwaves (EEG) and Maximum A Posteriori Model Adaptation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 743–752, Apr. 2007, doi: 10.1109/TPAMI.2007.1012.

[10] E. D. Übeyli, "Combined Neural Network Model Employing Wavelet Coefficients for EEG Signals Classification," *Digital Signal Processing*, vol. 19, no. 2, pp. 297–308, Mar. 2009, doi: 10.1016/j.dsp.2008.07.004.

[11] T. Gandhi, B. K. Panigrahi, M. Bhatia, and S. Anand, "Expert Model for Detection of Epileptic Activity in EEG Signature," *Expert Systems with Applications*, vol. 37, no. 4, pp. 3513–3520, Apr. 2010, doi: 10.1016/j.eswa.2009.10.036.

[12] Y. Liu, O. Sourina, and M. K. Nguyen, "Real-Time EEG-Based Human Emotion Recognition and Visualization," in *2010 International Conference on Cyberworlds*, Singapore, Singapore: IEEE, Oct. 2010, pp. 262–269. doi: 10.1109/CW.2010.37.

[13] U. Orhan, M. Hekim, and M. Ozer, "EEG Signals Classification Using the \emph{K}-Means Clustering and a Multilayer Perceptron Neural Network Model," *Expert Systems with Applications*, vol. 38, no. 10, pp. 13475–13481, Sep. 2011, doi: 10.1016/j.eswa.2011.04.149.

[14] A. Delorme *et al.*, "EEGLAB, SIFT, NFT, BCILAB, and ERICA: New Tools for Advanced EEG Processing," *Computational Intelligence and Neuroscience*, vol. 2011, no. 1, p. 130714, 2011, doi: 10.1155/2011/130714.

[15] Y. Liu and O. Sourina, "Real-Time Subject-Dependent EEG-Based Emotion Recognition Algorithm," *Transactions on Computational Science XXIII: Special Issue on Cyberworlds*. Springer, Berlin, Heidelberg, pp. 199–223, 2014. doi: 10.1007/978-3-662-43790-2_11.

[16] M. Ali, A. H. Mosa, F. Al Machot, and K. Kyamakya, "EEG-based Emotion Recognition Approach for e-Healthcare Applications," in *2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN)*, Vienna, Austria: IEEE, Jul. 2016, pp. 946–950. doi: 10.1109/ICUFN.2016.7536936.

[17] B. M. Turner, C. A. Rodriguez, T. M. Norcia, S. M. McClure, and M. Steyvers, “Why More Is Better: Simultaneous Modeling of EEG, fMRI, and Behavioral Data,” *NeuroImage*, vol. 128, pp. 96–115, Mar. 2016, doi: 10.1016/j.neuroimage.2015.12.030.

[18] Y. Li *et al.*, “A Novel Bi-Hemispheric Discrepancy Model for EEG Emotion Recognition,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 13, no. 2, pp. 354–367, Jun. 2021, doi: 10.1109/TCDS.2020.2999337.

[19] A. T. Gifford, K. Dwivedi, G. Roig, and R. M. Cichy, “A Large and Rich EEG Dataset for Modeling Human Visual Object Recognition,” *NeuroImage*, vol. 264, p. 119754, Dec. 2022, doi: 10.1016/j.neuroimage.2022.119754.

[20] S. Katsigiannis and N. Ramzan, “DREAMER: A Database for Emotion Recognition Through EEG and ECG Signals From Wireless Low-cost Off-the-Shelf Devices,” *IEEE J. Biomed. Health Inform.*, vol. 22, no. 1, pp. 98–107, Jan. 2018, doi: 10.1109/jbhi.2017.2688239.

[21] Wei-Long Zheng and Bao-Liang Lu, “Investigating Critical Frequency Bands and Channels for EEG-Based Emotion Recognition with Deep Neural Networks,” *IEEE Trans. Auton. Mental Dev.*, vol. 7, no. 3, pp. 162–175, Sep. 2015, doi: 10.1109/TAMD.2015.2431497.

[22] R.-N. Duan, J.-Y. Zhu, and B.-L. Lu, “Differential Entropy Feature for EEG-based Emotion Classification,” in *2013 6th International IEEE/EMBS Conference on Neural Engineering (NER)*, San Diego, CA, USA: IEEE, Nov. 2013, pp. 81–84. doi: 10.1109/NER.2013.6695876.

Appendices

The code for the pipeline in Section 2.1 can be found at https://github.com/trumpeter122/EEG-Emo-Rec-Pipeline.

The code for the mobile client in Section 2.2.3 can be found in https://github.com/trumpeter122/EEG-Emo-Rec-Client-Mobile.